SIMATS ENGINEERING

COMMON ASSESSMENT

UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)

Course Name: Object Oriented Analysis and Design

Course Code: CSA1109

Assignment Weightage: 100%

Total Marks: 100

Name: Akash A

Reg: 192373015

GitHub: https://github.com/akkurthiakash/CSA1109

Code of Conduct

I, Akash A, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Akash

1. Problem Analysis and Requirement Summary

1.1 Problem Statement

The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is designed to assist emergency teams during disasters such as floods, earthquakes, fires, and landslides. The system uses autonomous drones equipped with cameras and sensors to collect real-time information about affected areas, locate victims, monitor environmental conditions, and transmit data to a central command system.

By combining autonomous aerial reconnaissance with a centralized command-and-control platform, ADDRRS shortens the time between a disaster being reported and victims being located, enabling emergency coordinators to deploy limited rescue resources where they are needed most.

1.2 Functional Requirements

- Register and manage disaster incidents.
- Define and manage disaster zones.
- Register and monitor drones.
- Assign drones to rescue missions.
- Plan and update drone routes.
- Collect images, videos, GPS locations, and sensor readings.
- Locate and record victims.
- Monitor drone status in real time.
- Generate emergency alerts.
- Coordinate emergency teams and resources.
- Handle drone failures and communication loss.

1.3 Non-Functional Requirements

- Real-time communication.
- High reliability and availability.
- Secure access for authorized personnel.
- Scalability for multiple disasters and drones.
- Fast response time.
- Maintainability and flexibility.
- Fault tolerance and failure handling.

2. Actor Identification

The table below lists the primary and secondary actors that interact with ADDRRS, along with their responsibilities.

Actor	Responsibility
Emergency Coordinator	Registers incidents and manages rescue operations
Drone Operator	Monitors and controls drones when required
Rescue Team	Performs physical rescue operations
System Administrator	Manages users, drones, and system configuration
Autonomous Drone	Collects data and performs assigned missions
Central Command System	Processes and manages disaster information

Actor	Responsibility
Victim	Person identified as requiring rescue
Sensor System	Provides environmental readings
Notification Service	Sends emergency alerts and notifications

2. Use Case Descriptions

2.1 Major Use Cases

- Register Disaster Incident
- Define Disaster Zone
- Register Drone
- Assign Drone to Mission
- Plan Drone Route
- Launch Drone
- Monitor Drone
- Collect Sensor Data
- Capture Images/Videos
- Detect/Locate Victim
- Generate Alert
- Coordinate Rescue Team
- Manage Resources
- Handle Drone Failure

- Complete Rescue Mission

2.2 Example Use Case: Assign Drone to Rescue Mission

Field	Description
Use Case	Assign Drone to Rescue Mission
Primary Actor	Emergency Coordinator
Precondition	Disaster incident and drone are registered.
Main Flow	1. Coordinator selects a rescue mission. 2. System displays available drones. 3. Coordinator selects a suitable drone. 4. System assigns the drone and confirms the assignment.
Postcondition	The selected drone is assigned to the rescue mission and its status is updated to Assigned.

2.3 Use Case Diagram

The Use Case Diagram below shows the ADDRRS system boundary with the major use cases inside it and the external actors placed outside the boundary, connected through their respective associations.

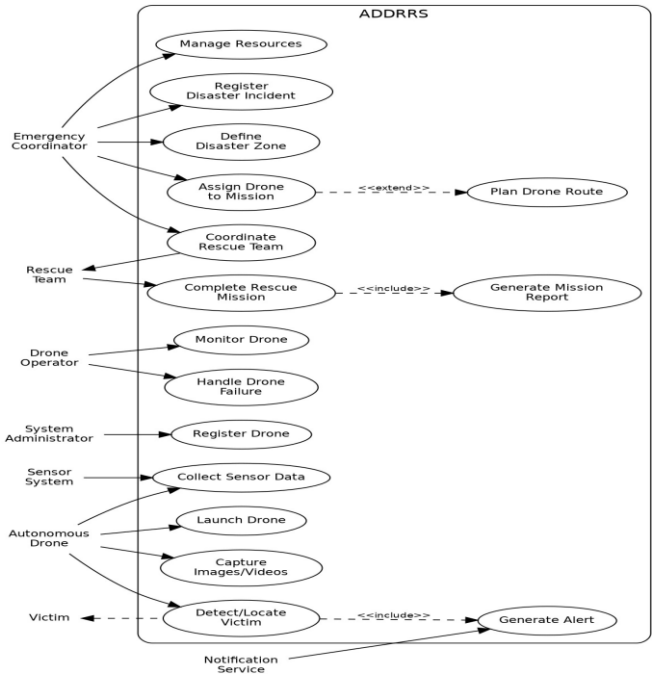


Figure 1: Use Case Diagram for ADDRRS

3. Object and Class Identification

3.1 Major Objects

Disaster, DisasterZone, Drone, DroneOperator, RescueMission, Victim, EmergencyTeam, SensorData, Alert, Resource, Route, MissionReport, EmergencyCoordinator.

3.2 Major Classes

Class	Important Attributes	Important Methods
Drone	droneId, status, location, batteryLevel	launch(), land(), move(), collectData()
Disaster	disasterId, type, severity, location	register(), updateStatus()
DisasterZone	zoneId, boundary, riskLevel	defineZone(), updateRisk()
RescueMission	missionId, status, priority	assignDrone(), startMission(), completeMission()
Victim	victimId, location, condition	updateCondition(), requestRescue()
EmergencyTeam	teamId, teamName, status	deploy(), performRescue()
SensorData	dataId, type, value, timestamp	collect(), transmit()
Alert	alertId, type, severity, message	generate(), send()
Route	routeId, startPoint, destination	calculate(), updateRoute()
Resource	resourceId, type, quantity	allocate(), release()
MissionReport	reportId, summary, timestamp	generate(), store()

4. Class Diagram

The Class Diagram shows the classes identified above together with their key relationships: Disaster to DisasterZone and RescueMission, RescueMission to Drone / EmergencyTeam / Victim / MissionReport, and Drone to SensorData / Route / Alert. Associations, aggregations, and dependencies are used to model these relationships, and a DroneManager and EmergencyCoordinator control class are included to show responsibility assignment.

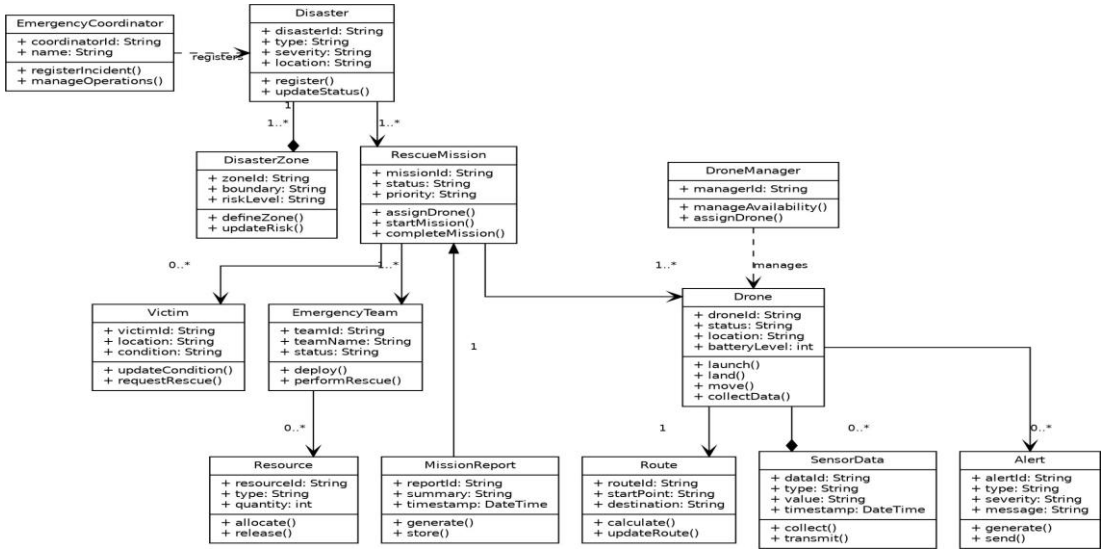


Figure 2: Class Diagram for ADDRRS

5. Sequence / Interaction Diagrams

Two sequence diagrams are presented below to illustrate representative interaction flows in the system.

5.1 Sequence Diagram 1: Assign Drone to Rescue Mission

This diagram shows how the Emergency Coordinator selects a mission, the Command System requests and receives available drones from the Drone Manager, assigns a drone, and confirms the assignment back to the coordinator.

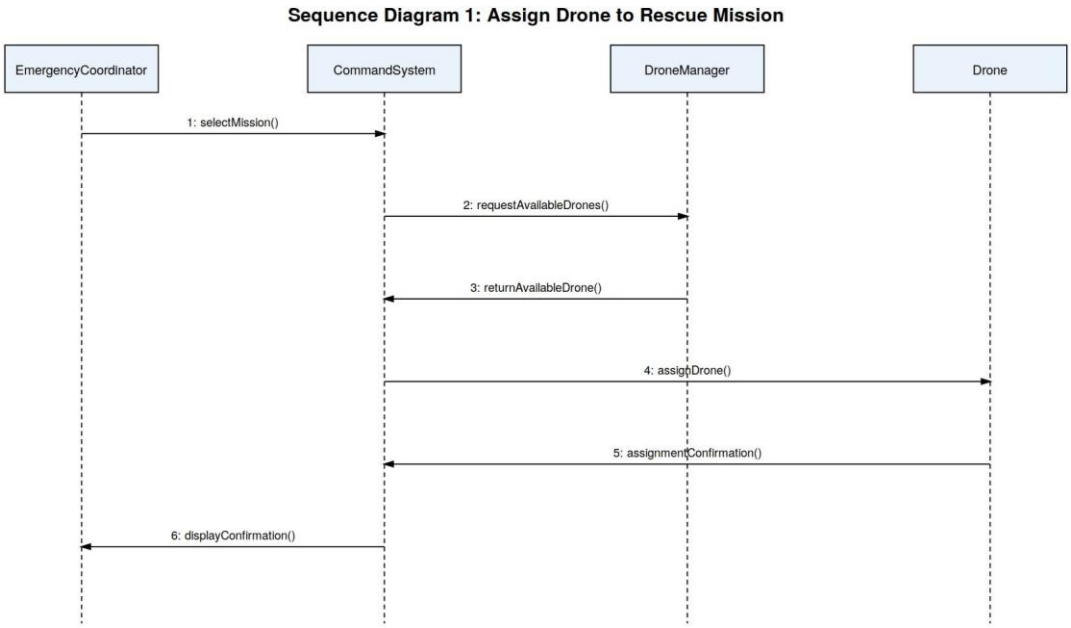


Figure 3: Sequence Diagram - Assign Drone to Rescue Mission

5.2 Sequence Diagram 2: Autonomous Victim Detection

This diagram shows how a drone captures image data through its camera/sensor, forwards it to the Command System for analysis by the Victim Detection Module, and how a detected victim triggers an alert that is routed to the Rescue Team through the Alert Manager.

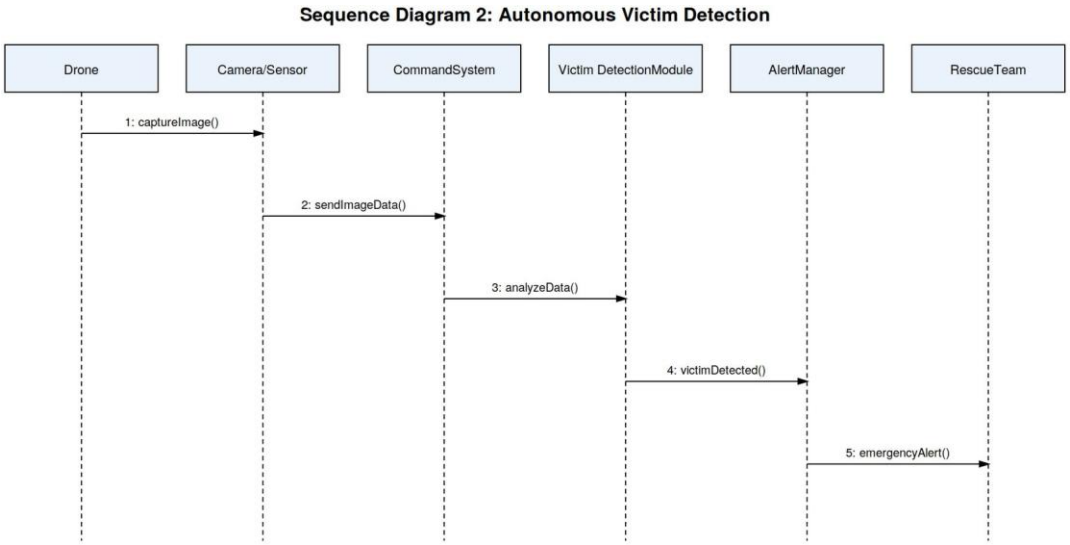
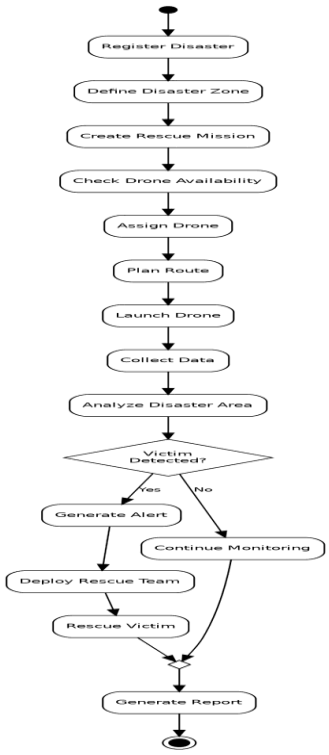


Figure 4: Sequence Diagram - Autonomous Victim Detection

6. Activity Diagram

The Activity Diagram below models the end-to-end Rescue Mission workflow, from disaster registration through drone assignment, data collection, victim detection, and mission-report generation. A decision node captures the branch between an area with no detected victim (continued monitoring) and one where a victim is detected .

Figure 5: Activity Diagram - Rescue Mission Workflow



7. State Transition Diagram

The Drone class is the most suitable object for a State Transition Diagram, since a drone moves through a well-defined lifecycle during a mission. The possible states are: Registered, Available, Assigned, Ready, In Flight, Returning, Mission Completed, Low Battery, Communication .

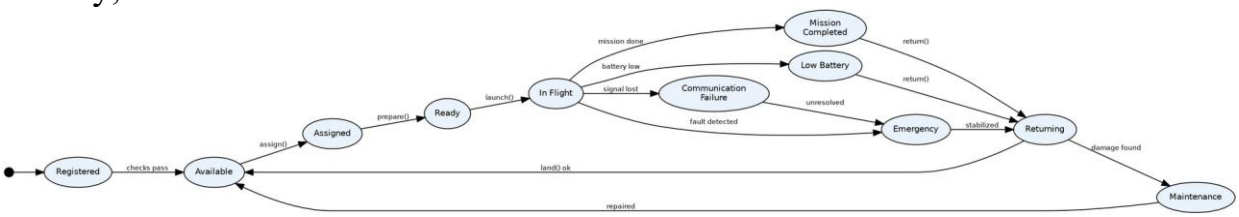


Figure 6: State Transition Diagram for Drone

8. Package Diagram

ADDRRS is organized into the following packages to improve modularity and maintainability: User Management, Disaster Management, Drone Management, Mission Management, Victim Management, Sensor & Data Management, Alert Management, Resource Management, Route Planning, Reporting, and Communication.

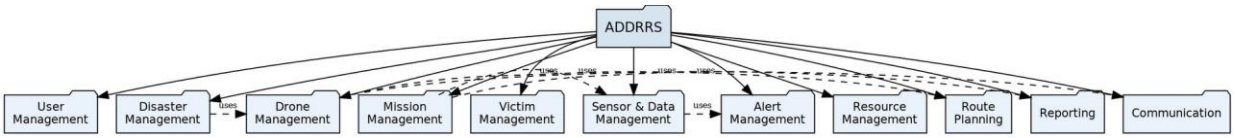
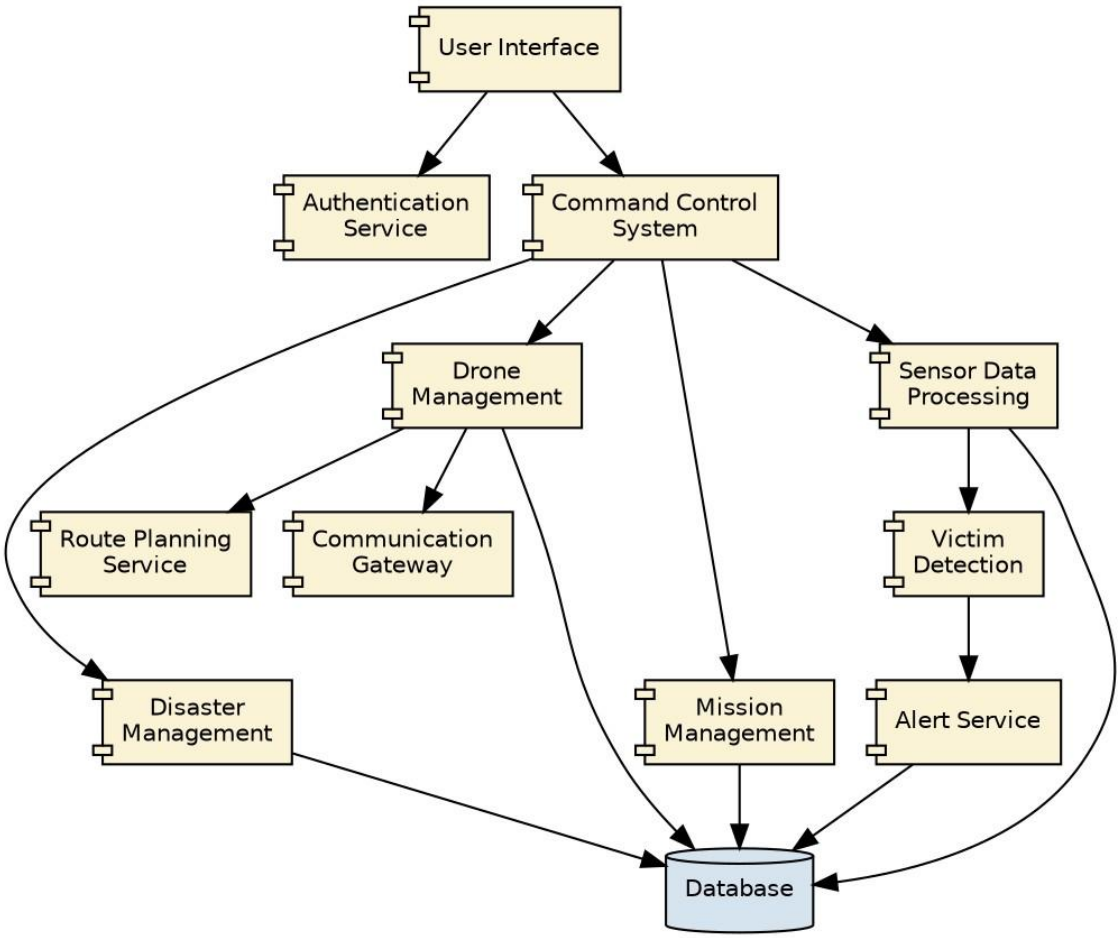


Figure 7: Package Diagram for ADDRRS

9. Component Diagram

The major components of ADDRRS are the User Interface, Authentication Service, Command Control System, Disaster Management, Drone Management, Mission Management, Route Planning Service, Sensor Data Processing, Victim Detection, Alert Service, Database, and Communication Gateway. The diagram below shows how the Command Control System coordinates the domain components, which in turn persist data to the shared Database.

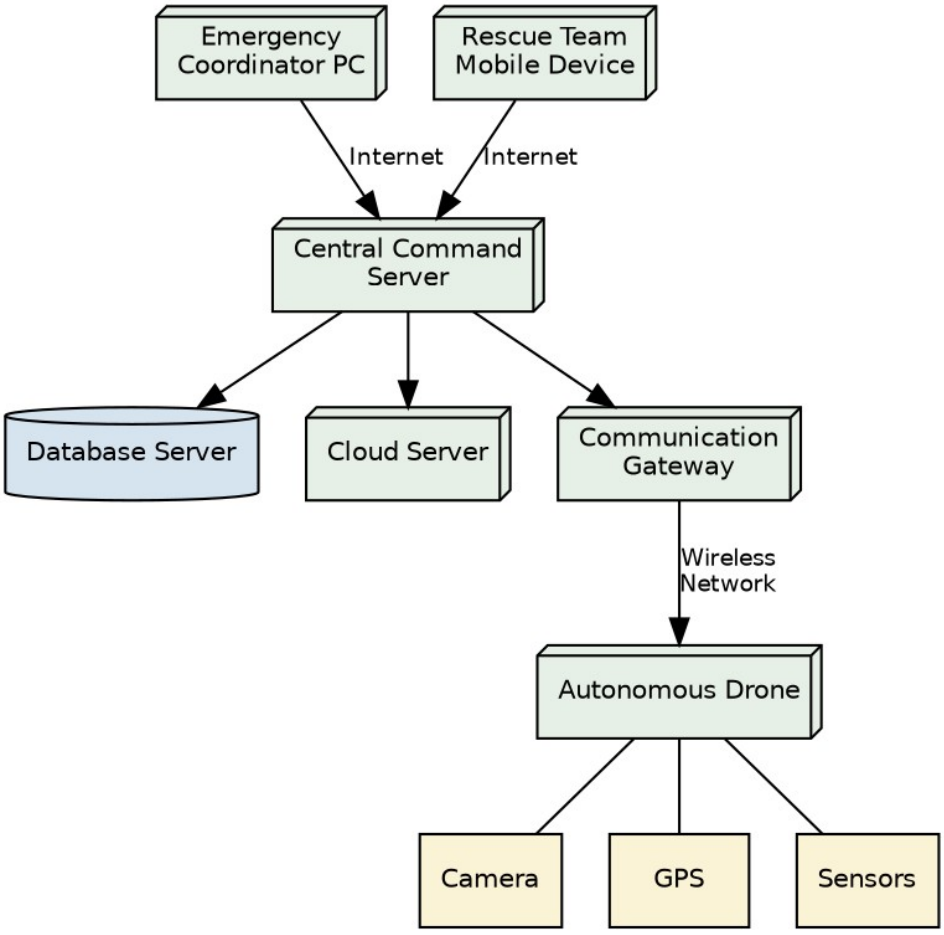
Figure 8: Component Diagram for ADDRRS



10. Deployment Diagram

The Deployment Diagram shows the physical nodes on which ADDRRS runs: the Emergency Coordinator Computer and Rescue Team Mobile Device connect over the Internet to the Central Command Server, which is linked to the Database Server and Cloud Server. The Central Command Server communicates with Autonomous Drones (equipped with Camera, GPS, and Sensors) through a Communication Gateway over a wireless network.

Figure 9: Deployment Diagram for ADDRRS



11. Object Responsibility Analysis

Object/Class	Responsibility
Drone	Execute autonomous flight and collect information
RescueMission	Manage mission lifecycle
Disaster	Store disaster information
DisasterZone	Represent affected geographical area
Route	Calculate and manage drone routes
Victim	Maintain victim information
SensorData	Store environmental readings
Alert	Manage emergency notifications
EmergencyTeam	Perform rescue activities
Resource	Manage rescue resources
MissionReport	Generate mission completion information
DroneManager	Manage drone availability and assignments

11.1 Responsibility Assignment

The design follows high cohesion by giving each class a focused responsibility (for example, Route only calculates and updates routes, and MissionReport only generates and stores reports) and low coupling by minimizing unnecessary dependencies between classes -- coordination logic is centralised in RescueMission and DroneManager rather than scattered across every domain class.

12. Design Axioms, Principles and Design Patterns

12.1 Design Principles

Principle	**Application in ADDRRS**
High Cohesion	Each class performs closely related operations (e.g. Drone only handles flight and data collection).
Low Coupling	Classes depend on interfaces/abstractions rather than unnecessary concrete implementations.
Single Responsibility Principle	Each class has one major responsibility (e.g. Alert only manages notifications).
Open/Closed Principle	New drone or sensor types can be added without major changes to existing code.
Encapsulation	Internal data such as drone status and battery level is protected through appropriate methods.

12.2 Suitable Design Patterns

Pattern	**Application**
Observer Pattern	Real-time drone status and alert notifications
Strategy Pattern	Different route-planning algorithms
Factory Pattern	Creating different drone/sensor objects
State Pattern	Managing drone states
Command Pattern	Sending commands to autonomous drones
Singleton Pattern	Central command/control manager, if a single shared instance is required

13. Assumptions and Constraints

13.1 Assumptions

- Only authorized personnel can access the system.
- Drones have GPS capability.
- Drones have cameras and environmental sensors.
- A communication network is available in most operational areas.
- Emergency teams have suitable communication devices.
- The central system maintains a database.

13.2 Constraints

- Limited drone battery capacity.
- Possible communication failures.
- Poor weather conditions may affect drone operation.
- GPS signals may become unavailable.
- Network bandwidth may be limited.
- System must handle multiple drones simultaneously.
- Security must prevent unauthorized access.
- Real-time data processing is required.

14. Requirement-to-Use Case/Class Traceability

Requirement	Use Case	Class(es)
Register disaster	Register Disaster Incident	Disaster
Define affected area	Define Disaster Zone	DisasterZone
Manage drones	Manage Drone	Drone, DroneManager
Assign drone	Assign Drone	Drone, RescueMission
Plan route	Plan Route	Route
Collect sensor data	Collect Data	Drone, SensorData
Locate victims	Locate Victim	Victim, Drone
Generate emergency notification	Generate Alert	Alert
Coordinate rescue	Coordinate Rescue	EmergencyTeam, RescueMission
Manage resources	Manage Resources	Resource
Complete mission	Complete Mission	RescueMission, MissionReport

15. Implementation / Prototype and Result

15.1 Prototype Scope

A simple prototype can demonstrate the following screens/functions:

- Login page
- Disaster registration
- Drone registration
- Drone status dashboard
- Mission assignment
- Disaster-zone display
- Victim information
- Sensor-data monitoring
- Emergency alerts

15.2 Expected Result

The prototype should demonstrate that emergency personnel can register disasters, assign drones, monitor autonomous operations, receive collected information, identify victims, generate alerts, coordinate rescue teams, and complete rescue missions.

16. Reflection on Design Decisions and Learning Outcomes

The development of ADDRRS demonstrates the application of Object-Oriented Analysis and Design principles to a real-world emergency-response problem. UML diagrams helped identify system actors, objects, classes, relationships, workflows, interactions, states, components, and deployment nodes.

The project provided practical understanding of:

- Object identification and classification.
- Use Case Modelling.
- UML diagram development.
- Class and relationship identification.
- Responsibility assignment.
- Object-oriented design principles.
- Design patterns.

GitHub Upload

https://github.com/akkurthiakash/CSA1109